

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 11

JavaScript Workflow und Tools

11.1 Allgemeines zu Workflows

JavaScript hat sich in den letzten Jahren zur dominanten Sprache für Web-Frontend-Entwickler*innen entwickelt. Dabei haben sich auch die Arbeitsweisen stark diversifiziert. Von der Modifizierung einzelner Elemente mittels nativem DOM, über die Arbeit mit klassischen Hilfsbibliotheken wie jQuery¹ (welches heute meist nur mehr in Legacy-Projekten verwendet wird) bis hin zum Einsatz moderner Frontend-Frameworks wie React, Vue.js, Angular oder Svelte – die Arbeitsweisen und Ansätze sind vielfältig. Durch die aufkommende Popularität von Node.js haben sich ebenfalls neue Möglichkeiten ergeben, die in den folgenden Abschnitten auch angeschnitten werden.

Ebenso haben sich im Bereich der Stylesheets neue Möglichkeiten aufgetan. Zwar bietet natives CSS mittlerweile mächtige Features, die früher Präprozessoren vorbehalten waren (z. B. CSS-Variablen oder Verschachtelung), doch in professionellen Projekten stoßen reine CSS-Dateien oft an organisatorische Grenzen. Hier helfen Workflows, um Stylesheets modular zu verwalten, automatisch für unterschiedliche Browser zu optimieren oder mathematische Operationen schon vor der Auslieferung zu berechnen.

Schließlich sind auch Performance und Codequalität Faktoren, die in modernen Webprojekten entscheidend sind. Schnelle Ladezeiten sind essentiell für eine gute User Experience (UX) und das Ranking in Suchmaschinen. Zu diesem Zweck müssen Style- und Script-Dateien technisch optimiert und verkleinert (minifiziert) werden. Parallel dazu gilt es, durch Coding-Standards (Linting) Flüchtigkeitsfehler zu vermeiden und sauberen, wartbaren Code zu erzeugen – eine Anforderung, die nicht nur in großen Teams wichtig ist.

Das Ziel solcher Workflows ist die Automatisierung: Sie sollen Entwickler*innen von repetitiven Aufgaben entlasten, damit der Fokus auf der eigentlichen Umsetzung liegen kann. Statt manuell Dateien zu verkleinern oder Syntax zu prüfen, übernehmen Tools diese Arbeit im Hintergrund. Dieses Kapitel zeigt auf, wie ein solcher Workflow beispielhaft aufgebaut werden kann.

¹<https://jquery.com/>

11.2 Node.js zur Workflow-Unterstützung

Wenn von Workflows gesprochen wird, geht es in der Regel immer auch um Automatisierung. Dinge sollen möglichst ohne wiederholendes Zutun des*der Entwicklers*Entwicklerin geschehen, damit diese sich auf den Kern ihrer Arbeit – das Entwickeln einer Webseite konzentrieren können. Um Automatisierung zu ermöglichen, benötigt es Tools, die bestimmte Aufgaben ausführen, etwa Dateien verarbeiten oder auf Änderungen in diesen zu überprüfen. Diese Aufgaben übernehmen ausführbare Dateien, engl. „binaries“, die in verschiedenen Sprachen geschrieben werden können. Populär sind etwa Tools in Ruby oder Python, Linux/Unix-Bash bzw. (Power-) Shell-Skripte, selbst Windows-Executables sind möglich. Sind Tools in derartigen Sprachen verfasst, ist es notwendig, die entsprechenden Laufzeitumgebungen auf dem eigenen Rechner installiert zu haben, etwa einen Ruby-Interpreter, um Ruby-Programme (sogenannte „Gems“) ausführen zu können. Dies ist jedoch nicht immer einfach, da die verschiedenen Plattformen unterschiedlich auf diversen Betriebssystemen unterstützt sind. Windows-Executables lassen sich nur mit viel Aufwand auf Unix-System ausführen, auch umgekehrt muss die entsprechende Umgebung installiert werden, damit Bash-Skripte unter Windows ausführbar sind. Darüber hinaus ist es dann vielfach unumgänglich, die jeweiligen Sprachen zu beherrschen.

Zusätzlich zu den soeben genannten Umgebungen hat sich seit einigen Jahren noch eine weitere etabliert, die jedoch nicht immer sofort als solche erkannt wird: *Node.js*.

11.2.1 Node.js im Überblick

Node.js ist eine Plattform zur Entwicklung von serverseitigem JavaScript. Sie baut auf Googles V8 JavaScript-Laufzeitumgebung (engl. „runtime“) auf und ermöglicht es, JavaScript-Applikationen am Server auszuführen. Somit lassen sich – einfach ausgedrückt – komplette Webseiten mit JavaScript erstellen, so wie es mit anderen serverseitigen Sprachen wie etwa PHP möglich ist. Node.js baut dabei auf einem eventbasierten Workflow auf, d. h. ein Webserver, auf dem Node.js läuft, generiert einen Event, wenn eine neue Verbindung eingeht und ruft eine Funktion auf (sprich einen Event-Handler), die sich um diese Verbindung kümmert.

Der folgende Code startet einen Webserver mit Node.js und zeigt bei jedem Request „Hello World!“ an:

```
1 import {createServer} from "node:http";
2
3 const hostname = "127.0.0.1";
4 const port = 3000;
5
6 const server = createServer((request, response) => {
7   response.statusCode = 200;
8   response.setHeader("Content-Type", "text/plain");
9   response.end("Hello World");
10 });
11
12 server.listen(port, hostname, () => {
13   console.log(`Server running at http://${hostname}:${port}/`);
14 });
```

Der Code liegt in einer Datei namens `nodestserver.mjs`. Die Erweiterung kennzeichnet ein so genanntes ECMAScript-Modul, welches in modernen Node.js-Anwendungen zum Einsatz kommt.

ECMAScript Module

ECMAScript Module (ESM, Dateierweiterung `.mjs`) sind ein standardisiertes Modulsystem für JavaScript, das es ermöglicht, Code in separate Dateien zu organisieren und wiederzuverwenden. Sie nutzen die `import` und `export` Schlüsselwörter, um Funktionen, Objekte oder Werte zwischen Modulen auszutauschen. ESM bieten einen sauberen, zukunftssicheren Ansatz für die Strukturierung von JavaScript-Code, der sowohl in Browsern als auch in Node.js-Umgebungen funktioniert.

Zunächst wird aus dem Node-Module (auch Package genannt) `node:http`, welches die Funktionalität enthält, um einen Webserver zu starten die Methode `createServer()` importiert. Die Angabe in geschweiften Klammern nennt man eine destrukturierende Zweisung² und importiert nur diese eine Methode und nicht das gesamte Modul.

Nach der Angabe des Hostnamens und eines Ports, auf dem der Server laufen soll wird mit `createServer()` ein neues Webserver-Objekt erstellt. Als Parameter wird eine sogenannte Callback-Funktion übergeben (hier in Form einer Arrow-Funktion mit `() => { ... }`, es könnte aber auch als anonyme Funktion der Form `function () { ... }` geschrieben werden), die immer dann aufgerufen wird, wenn eine Verbindung zum Webserver hergestellt wird, d. h. wenn jemand die Seite im Browser öffnet. In diesem Fall wird der Statuscode „200 – OK“ gesetzt und der Inhalt als reines Textdokument definiert. Danach wird „Hello World!“ ausgegeben und die Ausgabe beendet. Über den Aufruf von `listen()` hört der Webserver auf dem angegebenen Port. Im Browserfenster kann nun über `http://127.0.0.1:3000/` oder alternativ `http://localhost:3000/` diese Seite geöffnet werden. In der Konsole wird der URL zum Server mit Port angegeben.

Damit dies alles jedoch überhaupt funktioniert, muss zunächst Node.js installiert und dieses Script damit ausgeführt werden.

11.2.2 Node.js installieren

Damit mit Node.js gearbeitet werden kann, muss es am eigenen Rechner installiert werden (damit später Applikationen auf einem Webserver ausgeführt werden können, muss es selbstredend auf dem Webserver ebenso installiert sein). Zur Installation wird von der offiziellen Website³ unter [Download](#) das Installationsprogramm („Prebuilt Installer“) für die aktuellste Version – derzeit 25.4.0 – für das jeweilige Betriebssystem heruntergeladen und im Anschluss installiert.

Bei der Installation unter Windows wird abgefragt, ob Tools für native Module installiert werden sollen. Diese sind nötig, um für manche Pakete, entsprechende ausführbare Dateien zu kompilieren. Wählen Sie diese Option daher aus (siehe Abbildung 11.1).

Alternativ können auch Package-Manager unter den jeweiligen Betriebssystemen verwendet werden, um den Installer nicht manuell herunterladen zu müssen:

²<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring>

³<https://nodejs.org/>

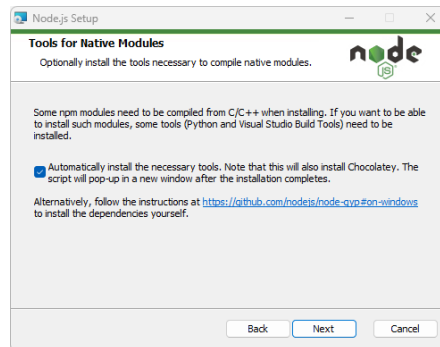


Abbildung 11.1: Bei der Installation sollen die Werkzeuge für native Module mitinstalliert werden, um später auch zuverlässig jedes Node-Package hinzufügen zu können.

- Windows mit *winget*: `winget install OpenJS.NodeJS`
- macOS mit *Homebrew*: `brew install node`

Um zu überprüfen, ob die Installation erfolgreich war, kann unter Windows die Eingabeaufforderung oder PowerShell bzw. unter macOS der Terminal geöffnet werden. Dort steht nun der Befehl `node` zur Verfügung, über den die Node.js Laufzeitumgebung gestartet wird. Durch `node -v` kann zunächst die installierte Version angezeigt werden:

```
> node -v
```

Die Ausgabe sieht wie folgt aus:

```
v25.4.0
```

Um nun das Webserver-Beispiel aus Abschnitt 11.2.1 auszuführen, wird der Name der JavaScript-Datei, die den Code enthält (hier `nodestserver.mjs`), als Argument an den `node`-Befehl übergeben:

```
> node nodestserver.mjs
```

11.2.3 Node.js Packages

Neben der Möglichkeit, skalierbare Netzwerkanwendungen zu erstellen, bietet Node.js noch ein zweites, großes Potential: Packages, auch Module genannt. Im Webserver-Beispiel in Abschnitt 11.2.1 wurde in Zeile 1 ein solches Package, nämlich `http` (im Namensraum `node`) eingebunden. Es enthält alle Funktionalitäten, um den Webserver auszuführen. Ebenso wie dieses Modul gibt es eine Vielzahl von der Community erstellten Packages, die unterschiedlichste Funktionalitäten vordefiniert zur Verfügung stellen. Einige davon sind für Workflow-Tätigkeiten konzipiert und werden in den folgenden Abschnitten im Detail erläutert.

Packages verwalten mit npm

Um Packages zu installieren, verwalten, aktualisieren oder auch zu entfernen, dient *npm*, der Node.js Package Manager. Gestartet als Zusatzprojekt ist dieser im Installationspaket von Node.js enthalten und kann direkt in der Kommandozeile bzw. im Terminal

verwendet werden. Um die aktuell installierte Version abzufragen (die übrigens nicht mit der Node.js Version übereinstimmt, da es sich um ein separates Projekt handelt), wird `npm -v` eingetippt:

```
> npm -v
```

Die Ausgabe der aktuellen Version lautet dann:

11.8.0

npm kann Packages global und lokal installieren. Globale Packages gelten für die gesamte Node.js-Installation und sind unabhängig von Projekten immer aufrufbar, d. h. sie agieren wie Commandline-Tools. Lokale Packages sind nur in dem Projekt (sprich Verzeichnis) verfügbar, in dem sie installiert werden.

Packages global installieren

Um ein Package oder Modul global zu installieren, kommt die folgende Syntax zur Anwendung:

```
> npm install -g <package>
```

Die Option `-g` stellt die globale Installation sicher, `<package>` wird mit dem jeweiligen Paketnamen ersetzt. Dieser Aufruf installiert das Package **sass** (eine JavaScript-Implementierung für den CSS-Präprozessor Sass, siehe Abschnitt 11.3) global:

```
> npm install -g sass
```

Packages lokal installieren

Um ein Package nur für das aktuelle Projekt zu installieren, wird folgender Aufruf eingesetzt:

```
> npm install <package>
```

`package` symbolisiert erneut das zu installierende Paket, eine Option ist keine nötig. Der folgende Aufruf installiert **sass** für das aktuelle Projekt, also *nicht* global:

```
> npm install sass
```

Die lokale Installation erzeugt einen Ordner **node_modules** im aktuellen Verzeichnis, in den das Modul und alle seine Abhängigkeiten (engl. „dependencies“) installiert werden.

Lokale Packages verwalten

Um die genauen Versionsnummern der installierten Packages zu verwalten, kann eine Datei namens **package.json** erstellt werden. Sie enthält eine Auflistung aller Module, die vom aktuellen Projekt benötigt werden. Dabei wird unterschieden, ob ein Package für die Ausführung, d. h. für den Produktivbetrieb oder nur für die Entwicklung vorausgesetzt wird.

Um eine derartige Datei zu erzeugen, muss folgender Aufruf in der Kommandozeile getätigt werden:

```
> npm init
```

Es folgt eine kurze Abfrage verschiedener Details zum Projekt wie dessen Name, Version und Beschreibung. Die meisten Optionen können so bestätigt werden, wie sie vorgeschlagen werden. Wer die Abfrage ganz umgehen möchte, gibt gleich `npm init -yes` ein. Um benötigte Packages anzugeben, werden diese lokal mit `npm install` installiert, dies wird in der Datei `packages.json` unter `dependencies` vermerkt. Um etwa das `sass`-Package zu installieren, wird folgendes aufgerufen:

```
> npm install sass
```

Eine `package.json`-Datei für ein Projekt das `sass` benötigt, könnte so aussehen:

```
1 {
2   "name": "wef1vo-sass-example",
3   "version": "1.0.0",
4   "description": "Example for using Sass in your project",
5   "keywords": [
6     "sass",
7     "example",
8   ],
9   "license": "ISC",
10  "author": "Wolfgang Hochleitner",
11  "type": "module",
12  "main": "index.js",
13  "scripts": {
14    "test": "echo \"Error: no test specified\" && exit 1"
15  },
16  "dependencies": {
17    "sass": "^1.97.3"
18  }
19 }
20 }
```

Es ist ersichtlich, dass nicht nur der Package-Name sondern auch die Versionsnummer spezifiziert wird. Dies ermöglicht es, die Abhängigkeiten des eigenen Projekts besser reproduzierbar und somit mit anderen teilbar zu machen.

11.3 Stylesheet Präprozessoren am Beispiel von Sass

Mit dem Wissen über Node.js und seine Module kann nun ein erstes Beispiel für einen typischen Frontend-Workflow demonstriert werden. Im Zentrum dessen steht Sass – ein CSS-Präprozessor.

11.3.1 Sass Einführung

*Sass*⁴ – kurz für Syntactically Awesome Stylesheets – ist ein CSS-Präprozessor, der die Arbeit mit Stylesheets erleichtern und Schwachstellen von CSS ausmerzen soll. Die grundsätzliche Funktionsweise besteht darin, dass Sass eine eigene Syntax, die SCSS-Syntax⁵ (Sassy CSS), bereitstellt, in der Stylesheets geschrieben werden. Die SCSS-

⁴<https://sass-lang.com/>

⁵Sass kennt auch die von YAML abgeleitete Sass-Syntax, diese ist jedoch weniger weit verbreitet und wird hier nicht näher besprochen.

Syntax mit Endung `.scss` ist dabei ein Superset der CSS-Syntax, d. h. jede CSS-Datei ist automatisch eine SCSS-Datei, letztere besitzen aber noch zusätzliche Befehle, die es in CSS nicht gibt. Über ein Commandline-Tool – den Präprozessor – werden SCSS-Dateien dann in herkömmliche CSS-Dateien umgewandelt und sind somit für den Browser verwendbar.

Wurde Sass für Node.js (**sass**) wie in Abschnitt 11.2.3 global installiert, kann der Präprozessor wie folgt aufgerufen werden:

```
> sass input.scss output.css
```

Die Input-Datei `input.scss` wird vom Präprozessor verarbeitet, aus ihr wird die Datei `output.css` generiert.

Im Folgenden werden die wichtigsten Grundelemente von Sass gezeigt, anhand derer einerseits die Syntax erklärt und auch die Grundidee vermittelt werden sollen. Das Grundprinzip von Sass lautet dabei *DRY* – Don't repeat Yourself. Codeteile, die in CSS aufgrund der Syntax öfters wiederholt werden müssen (da es nicht anders möglich ist), sollen in Sass nur einmal definiert werden.

11.3.2 Variablen

Ein Grundproblem in CSS ist die ungewollte Wiederholung von Informationen. Ein typisches Beispiel ist die Definition von Farben. Werden in einem Projekt etwa Grundfarben für Schrift und Hintergrund festgelegt, so kommen diese in der Regel an vielen Stellen im Stylesheet bzw. den Stylesheets vor. Soll die Farbe im Verlauf des Projekts geändert werden, müssen alle Vorkommen gesucht und entsprechend geändert werden. Sass eliminiert dieses Problem durch die Einführung von *Variablen*. Diese werden mit einem `$`-Zeichen definiert und können beliebige Werte wie Farben oder Schriftarten aufnehmen.⁶

Im folgenden Beispiel werden drei Variablen für Schriftfarbe (`$main-color`) und Schrift (`$main-font`), sowie für die Hintergrundfarbe (`$bg-color`) definiert und diese bei der Formatierung des `<body>`- und der `<a>`-Elemente angewandt. Die Quelldatei liegt unter `_scss/main.scss`, die Zieldatei unter `css/main.css`.

```
1 $main-color: #bada55;
2 $main-font: system-ui, sans-serif;
3 $bg-color: rgb(41 49 51);
4
5 body {
6   font-family: $main-font;
7   color: $main-color;
8   background-color: $bg-color;
9 }
10
11 a:link, a:visited, a:hover, a:active {
12   color: $main-color;
13 }
```

Nun muss der CSS-Präprozessor aufgerufen werden:

⁶Anmerkung: CSS beherrscht, wie in Abschnitt 5.4 gezeigt, Custom Properties, die Präprozessor-Variablen ähnlich sind. Es bestehen jedoch erhebliche Unterschiede, die hier erläutert werden: <https://www.smashingmagazine.com/2018/05/css-custom-properties-strategy-guide/>.


```
sass _scss/main.scss css/main.css
```

Dies erzeugt die folgende CSS-Datei:

```
1 body {
2   font-family: system-ui, sans-serif;
3   color: #bada55;
4   background-color: rgb(41, 49, 51);
5 }
6
7 a:link, a:visited, a:hover, a:active {
8   color: #bada55;
9 }
```

Die Werte für Schriftfarbe und -art wurden an den entsprechenden Stellen eingesetzt und mussten nicht mehrfach geschrieben werden. Ebenso ist ersichtlich, dass der Sass-Präprozessor noch die ältere `rgb()`-Schreibweise mit Beistrichen erzeugt.

11.3.3 Verschachtelung

Sass erlaubt es, CSS-Selektoren zu verschachteln, sodass spezifischere Selektoren dabei entstehen, ohne dass wiederum Information verdoppelt wird. Im folgenden Beispiel werden Elemente einer Navigation – die ungeordnete Liste, die Listenelemente, sowie die enthaltenen Links – im `<nav>`-Element *verschachtelt*:

```
1 nav {
2   ul {
3     list-style: none;
4     padding: 0;
5   }
6
7   li {
8     display: inline-block;
9   }
10
11   a {
12     text-decoration: none;
13   }
14 }
```

Nachdem der Präprozessor wie zuvor gezeigt aufgerufen wurde, ergibt sich folgender CSS-Code:

```
1 nav ul {
2   list-style: none;
3   padding: 0;
4 }
5
6 nav li {
7   display: inline-block;
8 }
9
10 nav a {
11   text-decoration: none;
12 }
```

Durch die Verschachtelung ist die hierarchische Zusammengehörigkeit der Elemente besser gegeben bzw. überhaupt ablesbar. In CSS liegen alle Selektoren auf einer Ebene.

Mittlerweile (seit Dezember 2023) unterstützen alle modernen Browser native CSS Verschachtelung (engl. „nesting“)⁷ [6]. Die Syntax dabei der von Sass sehr ähnlich. Sass zu verwenden bedeutet jedoch, dass das erzeugte CSS jedoch ohne Nesting entsteht, was maximale Kompatibilität – auch mit älteren Browsern – ermöglicht.

11.3.4 Partials und Module

Sass erlaubt die einfache Modularisierung von SCSS-Dateien. Um eine Datei mit nur kleinen SCSS-Teilen zu erstellen, wird dem Dateinamen ein Underscore (underscore) vorangestellt. Dadurch weiß der Präprozessor, dass es sich um ein sogenanntes *Partial* handelt und der Präprozessor versucht nicht, eine eigene CSS-Datei daraus zu generieren. Um das Partial zu verwenden, wird es mittels `@use`-Regel eingebunden. Damit behandelt Sass das Partial als Modul, alle Variablen oder Mixins (siehe Abschnitt 11.3.5) werden dann mit dem Namen des Partials vorangestellt angesprochen. Diese Art der Modularisierung ist sehr ähnlich zu ECMAScript-Modulen.

Im folgenden Beispiel, werden die drei Variablendeklarationen von Abschnitt 11.3.2 in eine eigene Datei namens `_variables.scss` ausgelagert:

```
1 $main-color: #bada55;
2 $main-font: system-ui, sans-serif;
3 $bg-color: rgb(41 49 51);
```

Diese Datei wird nun in `main.scss` ganz zu Beginn als Modul geladen, danach folgen die restlichen Regeln, in denen die Variablen mit dem Modulnamen `variables` vorangestellt angesprochen werden:

```
1 @use "variables";
2
3 body {
4   font-family: variables.$main-font;
5   color: variables.$main-color;
6   background-color: variables.$bg-color;
7 }
8
9 a:link, a:visited, a:hover, a:active {
10   color: variables.$main-color;
11 }
```

Es entsteht dieselbe CSS-Datei wie in Abschnitt 11.3.2. Bei der `@use`-Anweisung müssen sowohl der Underscore als auch die Dateiendung *nicht* angegeben werden.

Partials haben keine direkte Entsprechung in nativem CSS. die `@import`-Anweisung kann zwar auch mehrere CSS-Dateien in einer einbinden, dies erzeugt jedoch zusätzliche HTTP-Requests.

11.3.5 Mixins

Mixins sind Gruppen von SCSS-Deklarationen, die ähnlich wie eine Funktion einmal definiert und beliebig oft angewandt werden. Das Schlüsselwort lautet `@mixin`, danach folgt der Name des Mixins und in Klammern können – wie bei einer Funktion – Parameter angegeben werden, die beim Einbinden des Mixins angegeben werden.

⁷<https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Nesting>

Das folgende Mixin setzt den linken und rechten Innenabstand (Padding) auf den Wert des ersten Arguments, und den Wert des oberen und unteren Außenabstands (Margin) auf den Wert des zweiten Arguments und wendet es für alle Absätze an. Dazu wird die `@include`-Direktive benötigt:

```
1 @mixin padding-margin($pad, $mar) {  
2   padding-left: $pad;  
3   padding-right: $pad;  
4   margin-top: $mar;  
5   margin-bottom: $mar;  
6 }  
7  
8 p {  
9   @include padding-margin(10px, 20px);  
10 }
```

Nach Ausführung des CSS-Präprozessors entsteht folgender CSS-Code:

```
1 p {  
2   padding-left: 10px;  
3   padding-right: 10px;  
4   margin-top: 20px;  
5   margin-bottom: 20px;  
6 }
```

Dieses Mixin kann nun beliebig oft bei anderen Elementen mit unterschiedlichen Werten verwendet werden und erspart viel Schreibarbeit und/oder Code-Wiederholung.

Mixins existieren in nativem CSS bisher nur bedingt. Mit dem *CSS Functions and Mixins Module* [5] gibt es jedoch einen Entwurf, mit dem die Direktive `@function` in CSS eingeführt wird, um genau diese Funktionalität zu bieten. Derzeit funktioniert diese experimentell nur in Blink-basierten Browsern (Chrome, Edge, Opera).

11.3.6 Vererbung

Vererbung ist eine der nützlichsten Eigenschaften von Sass. Sie erlaubt es, Deklarationen, die in mehreren Selektoren vorkommen sollen, nur einmal anzugeben. Im Gegensatz zu Mixins sind keine Argumente möglich, dafür erzeugt Vererbung in der Regel weniger aufgeblähten CSS-Code.

Im nachfolgenden Beispiel werden drei Klassen für Nachrichten erzeugt. Eine generische `.message`-Klasse, die Rahmen und Schriftfarbe festlegt und eine `.status`- sowie `.error`-Klasse, die auf der vorigen aufbauen, aber den Rahmen grün bzw. rot definieren:

```
1 .message {  
2   border: 3px solid #ccc;  
3   padding: 10px;  
4   color: #333;  
5 }  
6  
7 .status {  
8   @extend .message;  
9   border-color: green;  
10 }  
11  
12 .error {  
13   @extend .message;
```

```
14 border-color: red;
15 }
```

Nach der Ausführung des Präprozessors entsteht folgender CSS-Code:

```
1 .message, .error, .status {
2   border: 3px solid #ccc;
3   padding: 10px;
4   color: #333;
5 }
6
7 .status {
8   border-color: green;
9 }
10
11 .error {
12   border-color: red;
13 }
```

Vererbung auf Selektorebene ist mit nativem CSS derzeit nicht realisierbar.

11.3.7 Weitere Funktionen

Sass wartet noch mit einer Reihe an weiteren Funktionen auf. Hierfür sei auf die Sass-Dokumentation [3] verwiesen.

11.4 Tasks automatisieren mit Gulp

Das Beispiel Sass hat gezeigt, dass mit Hilfe von Tools wie einem CSS-Präprozessor ein effizienterer Arbeitsstil erreicht werden kann. Das große Problem am gezeigten Workflow ist allerdings, dass der Präprozessor jedes Mal händisch in der Kommandozeile gestartet werden musste, sobald etwas an einer der SCSS-Dateien verändert wird:

```
sass _scss\main.scss css\main.css
```

Weiters skaliert diese Art Tools aufzurufen nur sehr schlecht. Sollen die Stylesheets im Anschluss auf Korrektheit überprüft und minifiziert werden, müssen noch zwei weitere Befehle auf der Commandline aufgerufen werden, sodass der Aufwand, die Tools zu starten, den Nutzen irgendwann übersteigt. Aus diesem Grund gibt es sogenannte Task-Runner, die derartige Aufgaben effektiv übernehmen. Ein solcher ist Gulp.

11.4.1 Gulp im Überblick

*Gulp*⁸ ist ein Task-Runner, öfters auch Build Tool genannt, das wiederholende und zeitintensive Aufgaben automatisiert. Diese Aufgaben werden in Tasks zusammengefasst, die in der Regel wie folgt ablaufen:

1. Quelldateien (z. B. `.js`, `.scss` etc.) aus dem Dateisystem laden.
2. Tools in der Pipeline hintereinander auf diese Dateien ausführen (z. B. Sass-Präprozessor, Minifier, Linter etc.).

⁸<https://gulpjs.com/>

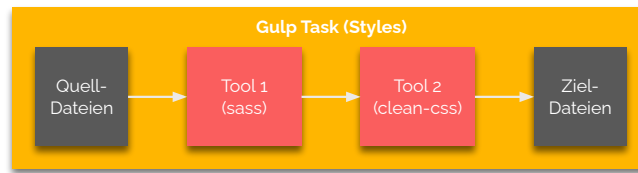


Abbildung 11.2: Ein exemplarischer Task in Gulp. Zuerst werden die Quelldateien geladen, darauf werden zwei Tools ausgeführt und das Ergebnis wird wieder gespeichert.

3. Veränderte Dateien wieder zurück ins Dateisystem (in der Regel mit neuem Namen) schreiben.

Abbildung 11.2 stellt diesen Ablauf für einen Styles-Task, bei dem zuerst Sass und dann ein Minifizierer (clean-css) ausgeführt werden, schematisch dar.

In den folgenden Abschnitten wird zunächst Gulp konfiguriert und dann für verschiedene Tasks eingesetzt.

11.4.2 Gulp vorbereiten

Gulp ist wie viele andere Tools als Node.js Modul verfügbar. Da es sich aber um ein explizites Commandline-Tool handelt, besteht es eigentlich aus zwei Teilen:

1. **Gulp-CLI:** Das Gulp Commandline-Interface (CLI) wird global installiert. Es ändert sich selten und hat nur die Aufgabe, das lokal installierte Gulp zu starten.
2. **Gulp:** Die lokale Gulp-Installation übernimmt die eigentliche Arbeit. Sie muss lokal in jedem Projekt separat installiert werden.

Gulp CLI installieren

Im ersten Schritt muss das Gulp CLI installiert werden. Dazu eine Commandline bzw. einen Terminal öffnen und folgendes eingeben:

```
> npm install -g gulp-cli
```

package.json initialisieren

Als nächstes muss das Verzeichnis des eigenen Webprojekts mit einer `package.json`-Datei initialisiert werden. Dazu in der Commandline ins entsprechende Verzeichnis wechseln und folgendes eingeben:

```
> npm init --yes
```

Dies erzeugt eine entsprechende Datei mit den Standardwerten.

Gulp lokal installieren

Nun muss die lokale Gulp-Installation vollzogen werden. Dazu wird Gulp mit dem Parameter `--save-dev` oder kürzer `-D` zu den Development-Dependencies hinzugefügt. Es wird somit als Tool zur Entwicklung, nicht zur Produktion gekennzeichnet.

```
> npm install -D gulp
```

Gulpfile erzeugen

Die Tasks für Gulp werden als ECMAScript-Modul in einer Datei namens `gulpfile.mjs` angelegt. Diese wird im Projektverzeichnis leer (z. B. mit PhpStorm) erstellt. Nun können in dieser Datei Tasks angelegt werden. Dieser Prozess wird im folgenden Abschnitt erklärt.

11.4.3 Tasks definieren

Gulp Tasks werden in `gulpfile.mjs` definiert. Diese Datei wird wie ein ECMAScript-Modul behandelt. Ein Modul ist, wie schon in Abschnitt 11.2.1 gezeigt, eine Datei mit JavaScript-Code, deren Funktionen so aufbereitet werden können, dass sie woanders wiederverwendbar sind. Dazu existiert das Schlüsselwort **export**. Alle Werte und Funktionen, denen dieses Schlüsselwort vorangestellt wird, werden exportiert und können „von Außen“ verwendet werden, wenn das Modul eingebunden wird.

So verhält es sich auch mit Gulp-Tasks. Diese werden als einfache Funktionen definiert. Werden sie anschließend dem `exports`-Objekt zugewiesen, sind sie öffentlich und können von Gulp aus als Task aufgerufen werden. Geschieht dies nicht, sind sie private Tasks, die nur als Hilfs- oder Teiltasks innerhalb eines öffentlichen Tasks verwendet werden können.

Der folgende Code definiert einen öffentlichen Gulp-Task, der „Hallo Welt!“ auf der Console ausgibt und das prinzipielle definieren eines Tasks veranschaulicht:

```
1 export function test(cb) {  
2   console.log("Hallo Welt!");  
3   cb();  
4 }
```

Zunächst wird eine Funktion `test()` mit vorangestelltem **export** definiert, die über `console.log()` die Hallo-Welt-Botschaft ausgibt. Da Gulp-Tasks asynchron sind (mehr dazu im zweiten Semester), muss angedeutet werden, wann die Funktion mit ihren Aufgaben fertig ist. Dies geschieht entweder, wenn etwas mit **return** zurückgegeben wird, oder muss sonst – wie in diesem Beispiel – mit einem sogenannten „Error-first Callback“ geschehen. Dies ist eine Funktion, die aufgerufen wird, um den Abschluss der Aufgaben einer Funktion anzuzeigen. Sie wird hier `cb()` (kurz für Callback) genannt, kann aber prinzipiell jeden Namen tragen. Wird sie ohne Parameter aufgerufen, zeigt Gulp an, dass alles korrekt funktioniert hat. Mehr dazu in der Gulp Dokumentation⁹.

Um den Task auszuführen, wird er mit dem Schlüsselwort **gulp** in der Konsole eingegeben:

```
> gulp test
```

Das Ergebnis sieht wie folgt aus:

```
[12:00:00] Using gulpfile ...\\wef1vo-examples\\vo11\\gulp\\gulpfile.mjs  
[12:00:00] Starting 'test'...  
Hallo Welt!  
[12:00:00] Finished 'test' after 1.68 ms
```

⁹<https://gulpjs.com/docs/en/getting-started/async-completion/>

Komplexere Tasks verwenden für gewöhnlich ein Node.js-Modul, das eine bestimmte Aufgabe übernimmt, z. B. Stylesheet-Preprocessing. Daher sind in der Regel immer mehrere Schritte notwendig.

1. Das benötigte Node.js-Modul (Package) wird als Development-Dependency installiert: `npm install -D <package>`.
2. Das Package (Modul) in `gulpfile.mjs` mit `import` einbinden: `import name from "package";`.
3. Gulp selbst ebenfalls mit `import` destrukturierend einbinden, um auf dessen Methoden `src()` (Lesen von Quelldateien) und `dest()` (Schreiben in Zieldateien) zugreifen zu können.
4. Mittels `src()` die Quelldateien auswählen, die verarbeitet werden sollen.
5. Das Modul mit der `.pipe()`-Methode einbinden, sodass es auf die Quelldateien wirkt.
6. Das Ergebnis mit der `.pipe()`-Methode in ein durch `dest()` spezifiziertes Ziel schreiben.

Dieser Prozess wird nun anhand von mehreren Beispielen demonstriert.

11.4.4 JavaScript-Minifizierung mit gulp-terser

Eine einfache Art und Weise, die Ladegeschwindigkeit von Seiten zu erhöhen, ist die *Minifizierung* (engl. „minification“) von JavaScript-Dateien. Dabei werden alle unnötigen Leerzeichen entfernt und Variablennamen so kurz wie möglich gehalten. Ein Tool, das dies beherrscht, ist `terser`¹⁰, welches für Gulp als `gulp-terser`¹¹ verfügbar ist.

Es muss zunächst im Projektverzeichnis lokal als Development-Dependency installiert werden:

```
> npm install -D gulp-terser
```

Danach wird `gulpfile.mjs` editiert: zunächst wird den Konstanten `src` und `dest` das Gulp-Modul zugewiesen. Dies ist die bereits angesprochene destrukturierende Zuweisung und wählt von einem Modul (hier das Gulp-Modul) zwei Methoden (hier `src()` und `dest()`) aus. Dies entspricht dem kompletten Import des Gulp-Moduls und den anschließenden Zuweisungen `const src = gulp.src` und `const dest = gulp.dest`, ist jedoch effizienter, da eben nicht das komplette Modul geladen werden muss. Anschließend wird das `gulp-terser`-Modul eingebunden und dafür eine Task-Funktion namens `scripts` erstellt, die mit `export` exportiert wird:

```
1 import { src, dest } from "gulp";
2 import terser from "gulp-terser";
3
4 export function scripts() {
5   return src("_js/**/*.js")
6     .pipe(terser())
7     .pipe(dest("js"));
8 }
```

¹⁰<https://www.npmjs.com/package/terser>

¹¹<https://www.npmjs.com/package/gulp-terser>

In der Methode `scripts()` wird zunächst mittels `src()` eine Auswahl an Quelldateien getroffen. Das Argument `_js/**/*.js` ist das sogenannte *Globstar*-Muster. Es besagt, dass im Verzeichnis `_js` (der erste Teil des Musters) und in allen Unterverzeichnissen (der `**`-Teil) alle Dateien mit der Endung `.js` (der `*.js`-Teil) miteinbezogen werden sollen.

Über die `pipe()`-Methoden werden nun auf diese ausgewählten Dateien Funktionen aufgerufen. Zunächst ist dies `terser()`, die Hauptmethode des `gulp-terser`-Packages. Sie bewirkt das oben beschriebene Minifizierungs-Verhalten.

Nachdem die Datei durch diese Methode verändert wurde, muss der veränderte Inhalt wieder zurück ins Dateisystem geschrieben werden. Dies geschieht durch den Aufruf `dest()`, der ebenfalls mit `pipe()` angehängt wird. Hier wird nur der Verzeichnisname (in diesem Fall `js`) angegeben, der Dateiname ändert sich nicht. Wäre dies gewünscht, könnte dazu das Modul `gulp-rename` zuvor in die Pipeline gehängt werden.

Im Verzeichnis `_js` befindet sich als Ausgangsbasis die Datei `scripts.js` mit folgendem Inhalt:

```
1 "use strict";
2
3 const contentContainer = document.getElementById("content");
4
5 const tools = ["Node.js", "Gulp", "Sass", "clean-css"];
6
7 for (const toolName of tools) {
8   const card = document.createElement("p");
9   card.classList.add("tool-card");
10  card.textContent = `Tool: ${toolName}`;
11  contentContainer.appendChild(card);
12 }
```

Die Datei wird mit `defer` eingebunden und erzeugt nach dem Laden des Dokuments für jeden Eintrag im Array `tools` einen Absatz, der mit der Klasse `tool-card` formatiert wird und hängt diesen ins `<main>`-Element, welches durch die ID `content` gefunden wird. Nun wird der `scripts`-Task ausgeführt:

```
> gulp scripts
```

Das Ergebnis ist die Datei `js▶scripts.js` mit folgendem, einzeiligem Inhalt:

```
1 "use strict";const contentContainer=document.getElementById("content"),tools=["Node.js","Gulp","Sass","clean-css"];for(const t of tools){const n=document.createElement("p");n.classList.add("tool-card"),n.textContent=`Tool: ${t}`,contentContainer.appendChild(n)}
```

In diesem einfachen Beispiel ist der Gewinn durch die Minifizierung nicht sehr groß (261 Bytes vs. 345 Bytes), bei umfangreicheren Dateien zahlt sich dies in der Regel immer aus. Globale Variablen auf oberster Ebene, wie hier `tools` werden standarmäßig nicht minifiziert, da sie möglicherweise in einer anderen JavaScript-Datei oder im HTML angesprochen werden. Ist man sich sicher, dass dies nicht der Fall ist, kann über die Verwendung eines Options-Objekt als Parameter für `terser()` dies geändert werden: `terser({ toplevel: true })`.

11.4.5 Präprozessor für Styles mit Minifizierung und Sourcemaps

Auch für CSS-Dateien gibt es Gulp-Workflows. Im folgenden Beispiel wird zunächst der CSS-Präprozessor Sass gestartet, um SCSS-Dateien in CSS-Dateien zu konvertieren. Dann wird `clean-css` aufgerufen, um das Ergebnis zu minifizieren. Schließlich werden noch sogenannte Sourcemaps generiert, sodass in den Browser-Development-Tools nicht die minifizierten CSS-Dateien, sondern die unveränderten SCSS-Dateien betrachtet werden können.

Zunächst werden die vier Packages `sass`¹², `gulp-sass`¹³, `gulp-clean-css`¹⁴ sowie `gulp-sourcemaps`¹⁵ als Development-Dependencies installiert. Dies kann in einem Aufruf geschehen:

```
> npm install -D sass gulp-sass gulp-clean-css gulp-sourcemaps
```

In `gulpfile.mjs` werden alle vier Module eingebunden und eine neue Task-Funktion namens `styles()` erstellt und mit `export` gekennzeichnet. Das Modul `sass` wird dabei extra benötigt, da `gulp-sass` nicht auf einen Sass-Compiler festgelegt ist und dieser zunächst spezifiziert werden muss. Dieser Aufruf von `gulpSass` mit dem Compiler `dartSass` wird in der Variable `sass` hinterlegt und kann somit in `scripts()` einfacher aufgerufen werden.

```
1 import * as dartSass from "sass";
2 import gulpSass from "gulp-sass";
3 import cleanCSS from "gulp-clean-css";
4 import sourcemaps from "gulp-sourcemaps";
5
6 const sass = gulpSass(dartSass);
7
8 export function styles() {
9   return src("_scss/main.scss")
10     .pipe(sourcemaps.init())
11     .pipe(sass())
12     .pipe(cleanCSS())
13     .pipe(sourcemaps.write())
14     .pipe(dest("css"));
15 }
```

Als Quelldatei wird hier kein Globstar-Muster gesetzt, sondern direkt die Datei `_scss/main.scss` verwendet. Sie bindet mittels `@use`-Anweisungen weitere SCSS-Dateien ein, daher ist sie die einzige, die verarbeitet werden muss.

Im ersten `pipe()`-Aufruf werden die Sourcemaps mittels `sourcemaps.init()` initialisiert. Dies muss geschehen, bevor andere Operationen in der Pipeline ausgeführt werden, weiters müssen alle nun folgenden Plugins Sourcemaps unterstützen.

Der nächste Aufruf ist `sass()`, welches eine Datei `main.css` aus der SCSS-Datei erzeugt.

Dann geschieht durch den Aufruf von `cleanCSS()` die Minifizierung und schließlich werden die Sourcemaps mittels `sourcemaps.write()` in die CSS-Datei inline geschrieben. Das Ergebnis landet im Ordner `css`.

¹²<https://www.npmjs.com/package/sass>

¹³<https://www.npmjs.com/package/gulp-sass>

¹⁴<https://www.npmjs.com/package/gulp-clean-css>

¹⁵<https://www.npmjs.com/package/gulp-sourcemaps>

Die Ausgangsbasis der SCSS-Dateien sah wie folgt aus:

```

1  /* _variables.scss */
2  $red: rgb(255 0 0);
3  $gray-light: #f3f3f3;
4  $gray-dark: #888888;
5
6  /* _color.scss */
7  @use "variables";
8
9  h1 {
10     color: variables.$red;
11 }
12
13 /* _type.scss */
14 body {
15     font-family: system-ui, sans-serif;
16 }
17
18 /* _components.scss */
19 @use "variables";
20
21 .tool-card {
22     background-color: variables.$gray-light;
23     border: 1px solid variables.$gray-dark;
24     border-radius: 8px;
25     padding: 1rem;
26     margin-bottom: 0.5rem;
27     font-weight: bold;
28 }
29
30 /* main.scss */
31 @use "type";
32 @use "color";
33 @use "components";

```

Nun wird der Task in der Commandline aufgerufen:

```
> gulp styles
```

Das Ergebnis des Gulp Tasks in `css/main.css` sieht wie folgt aus:

```
1 body{font-family:system-ui,sans-serif}h1{color:red}.tool-card{background-color:#
  f3f3f3;border:1px solid #ddd;border-radius:8px;padding:1rem;margin-bottom:.5rem;
  font-weight:700}
2 /*# sourceMappingURL=data:application/json;charset=utf8;base64,eyJ2ZXJzaW9uIjozLCJz
  b3VyY2VzIjpbIl90eXB1bnNjc3MiLCJfY29sb3Iuc2NzcyIsIl92YXJpYWJsZXMuMuc2NzcyIsIl9jb21
  wb251bnRzLnNjc3MiXSwwbmFtZXMxO1tdLCJtYXBwaW5ncyI6IkFBQWUesSOFDRSxZQUFBLFBnBQWUesQ0
  FBQSxXQONGLEdBQOUstUNISSxJQOVOLFdBQOUsaUJERlcsUUNHWCxPQUFBLElBQWUesTUFBBQWUesLQUNBL
  GNBQWUesSUFDDQWUesRQUFBLEtBQOEsY0FBQWUesNQUUNBLF1BQWUeIlCJmawXlIjoibWFpbj5jc3MiLCJz
  b3VyY2VzQ29udGVudCI6WyJib2R5IHtccclxuICBmb250LWZhbWlseTogc3lzdGVtLXVpLCBzYW5zLXN1cm1
  m01xyXG59IiwicHVzZSBcInZhcm1hYmxlc1wi01xyXG5ccclxuaDEge1xyXG4gIGNvbG9y0iB2YXJpYW
  JsZXMuHJlZDtcclxufSIsIiRyZWQ6IHJnYyguNTUgMCAwKtTccclxuJGdyYXktbGlnaHQ6ICNmM2YzZ
  jM7XHJcbiRncmF5LWRhcms6ICNkZGRkZGQ7IiwicHVzZSBcInZhcm1hYmxlc1wi01xyXG5ccclxunRv
  b2wtY2FyZCB7XHJcbiAgYmFja2dyb3VuZC1jb2xvcjogdmFyaWFiVGZLiRncmF5LWxpZ2h001xyXG4
  gIGJvcmlRlcjogMkY6IHJnYmFja2dyb3VuZC1jb2xvcjogdmFyaWFiVGZLiRncmF5LWxpZ2h001xyXG4
  gM6IDhweDtcclxuICBwbWYWRkaW5nO1AxcmtV01xyXG4gIGhcmdbpb1ib3R0b206IDAuNXJlbTtccclxuI
  CBmb250LXdlawdodDogYm9sZDtcclxufSJdfQ== */
```

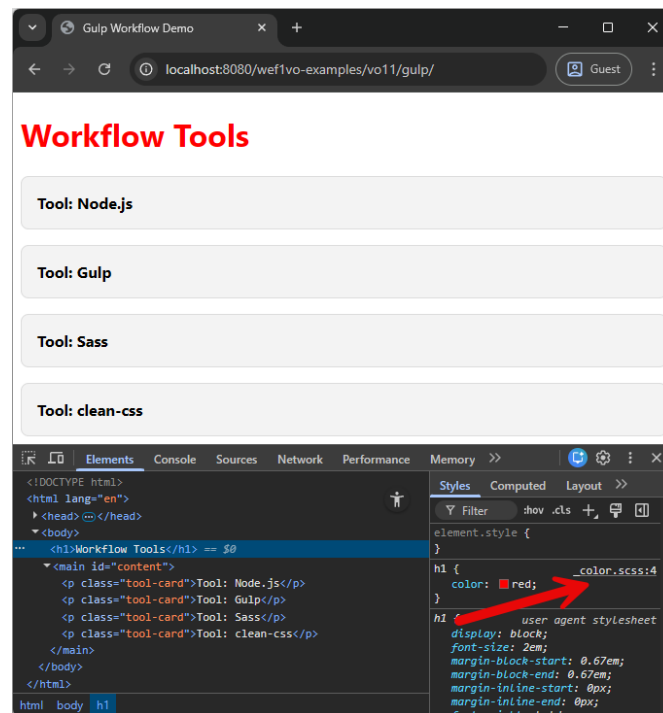


Abbildung 11.3: Die DevTools des Browsers zeigen an, dass die Definition der Regel für die `<h1>`-Überschrift aus der Datei `_color.scss` stammt, obwohl der Browser nur die minimierte CSS-Datei `main.css` zur Verfügung hat. Diese Zuordnung geschieht über die bereitgestellten Sourcemaps.

Der CSS-Code ist auf eine Zeile komprimiert, darunter befindet sich die Sourcemap, die Base64-kodiert (und daher nicht wirklich lesbar) ist. Dennoch zeigt sich in den DevTools des Browsers (Abbildung 11.3) ihre Auswirkung.

Beim Minifizieren wird sichtbar, dass clean-css hier wo immer möglich Zeichen spart:

- `rgb(255 0 0)` wird zu `red`,
- `#dddddd` wird zu `#ddd`,
- `0.5rem` wird zu `.5rem` und
- `bold` wird zu `700`.

11.4.6 Codequalität durch Linting

Automatisierte Tasks mit Gulp müssen nicht immer Änderungen an Dateien nach sich ziehen. Vielmehr können auch Tools aufgerufen werden, die nur lesend wirken. Ein solches Beispiel ist *Linting*, also die Ausführung eines Codeanalyse-Tools, das Fehler oder verletzte Coding-Standards anzeigt. Linter existieren für viele Sprachen, im Folgenden soll ESLint, ein Linter für JavaScript zum Einsatz kommen.

Zunächst muss das entsprechende Node.js Modul, nämlich `gulp-eslint-new`¹⁶ installiert werden:

¹⁶<https://www.npmjs.com/package/gulp-eslint-new>

```
> npm install -D gulp-eslint-new
```

Danach wird in `gulpfile.mjs` das Modul eingebunden, eine eigene Task-Funktion namens `lint()` erstellt und exportiert:

```
1 import esLintNew from "gulp-eslint-new";
2
3 export function lint() {
4   return src("_js/**/*.js")
5     .pipe(esLintNew())
6     .pipe(esLintNew.format())
7     .pipe(esLintNew.failAfterError());
8 }
```

Zunächst werden alle Dateien mit der Endung `.js` im Ordner `_js` und allen Unterverzeichnissen inkludiert. Dann wird die Funktion `esLintNew()` in der Pipeline aufgerufen. Dies speichert das Ergebnis des Lintings in der Variable `esLintNew` und macht es so für die Ausgabe oder auch allfällige andere Module verfügbar. `esLintNew.format()` gibt das Ergebnis auf der Console aus und `esLintNew.failAfterError()` weist den Prozess an, mit einem Fehler zu beenden, falls der Code nicht den angegebenen Standards entsprochen hat. Dies kann z. B. verwendet werden, um nachfolgende Tasks gar nicht erst auszuführen.

Mit diesen Einstellungen alleine kann ESLint jedoch nicht laufen. Es wird noch eine Konfiguration benötigt, die angibt, worauf überprüft werden soll. Diese kann u. a. in einer Datei namens `eslint.config.mjs` abgelegt werden. Ist sie vorhanden, liest sie ESLint automatisch aus und führt die Überprüfung danach durch. Diese Datei ist ebenfalls ein ECMAScript-Modul und kann etwa so aussehen:

```
1 export default [
2   {
3     rules: {
4       "semi": "error", // Error on missing semicolon
5       "indent": "warn", // Warning on inconsistent indentation
6       "prefer-const": "error" // Error when let is used instead of const
7     }
8   }
9 ];
```

Fehlende Strichpunkte werden als Fehler betrachtet, ist die Einrückung nicht korrekt, wird eine Warnung angezeigt. Ebenso wird ein Fehler gemeldet, wenn `let` anstatt `const` bei der Variablendeklaration verwendet wird.

Angenommen, die Datei `scripts.js` wäre wie folgt aufgebaut, d. h. in Zeile 8 würde `let` statt `const` verwendet, in Zeile 9 fehlte das Semikolon und Zeile 11 wäre falsch eingerückt ...

```
1 "use strict";
2
3 const contentContainer = document.getElementById("content");
4
5 const tools = ["Node.js", "Gulp", "Sass", "clean-css"];
6
7 for (const toolName of tools) {
8   let card = document.createElement("p");
9   card.classList.add("tool-card")
10  card.textContent = `Tool: ${toolName}`;
```

```
11   contentContainer.appendChild(card);
12 }
```

...so würde der Aufruf von

```
> gulp lint
```

folgenden Output erzeugen:

```
[12:00:00] Using gulpfile ...\\wefivo-examples\\vo11\\gulp\\gulpfile.mjs
[12:00:00] Starting 'lint'...
[12:00:00] ...\\wefivo-examples\\vo11\\gulp\\_js\\scripts.js
   8:9   error    'card' is never reassigned. Use 'const' instead    prefer-const
   9:36  error    Missing semicolon                                  semi
  11:1   warning   Expected indentation of 4 spaces but found 6       indent

X 3 problems (2 errors, 1 warning)
  2 errors and 1 warning potentially fixable -
    see https://www.npmjs.com/package/gulp-eslint-new/v/2.5.0#autofix

[12:00:00] 'lint' errored after 329 ms
[12:00:00] ESLintError in plugin "gulp-eslint-new"
Message:
  Failed with 2 errors
```

11.4.7 Testserver mit Live-Reload

Neben diversen Tools, die den Umgang mit Files vereinfachen, ist auch der Test im Browser unumgänglich. Auch hierbei gibt es Tools, die diese Angelegenheit vereinfachen. Dazu zählt das Package *Browsersync*, das automatisch einen lokalen Testserver erstellt und einen Browser zum Testen öffnet. Sind mehrere Browserfenster mit der Seite im Testserver geöffnet, können diese auch gleichzeitig aktualisiert werden.

Als ersten Schritt wird das Node.js Modul **browser-sync**¹⁷ wie bereits öfters demonstriert, als Development-Dependency installiert:

```
> npm install -D browser-sync
```

Anschließend wird in `gulpfile.mjs` eine Browsersync-Instanz mit `create()` erzeugt und eine Task-Funktion `server()` angelegt, in der Browsersync mit `init()` aufgerufen wird:

```
1 import browserSync from "browser-sync";
2
3 const bs = browserSync.create();
4
5 export function server(done) {
6   bs.init({
7     server: {
8       baseDir: "./"
9     }
10  });
11  done();
12 }
```

¹⁷<https://www.npmjs.com/package/browser-sync>

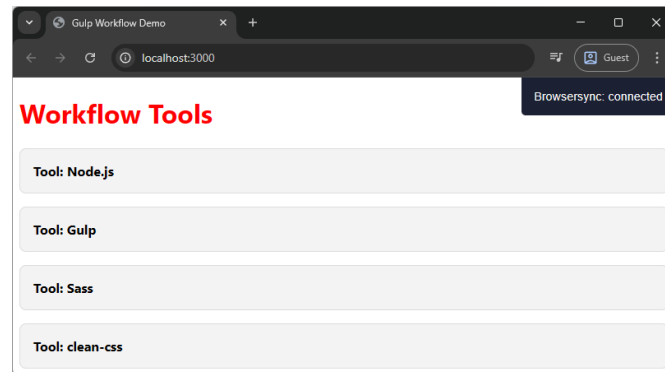


Abbildung 11.4: Nachdem der Gulp-Task ausgeführt wurde, öffnet sich der Browser und zeigt die Seite über einen lokalen Server auf Port 3000 (<http://localhost:3000/>) an.

Aufgerufen wird die Funktion `init()`, diese bekommt als Parameter ein JSON-Konfigurationsobjekt mit. In dessen Angabe `baseDir` steht der Pfad zu den HTML-Dateien. Befinden sich diese direkt auf der untersten Ebene, reicht `.` als Angabe. Liegen die Dateien im Unterverzeichnis `site`, müsste eben `site` angegeben werden. Die Funktion `done()` ist ein Error-first Callback (wie `cb()` beim Task „test“) und muss zum Schluss aufgerufen werden, um das Ende des Tasks anzudeuten, da hier anders als etwa bei „scripts“ oder „styles“ nichts zurückgegeben wird.

```
> gulp server
```

Über diesen Task wird nun ein lokaler Server auf Port 3000 gestartet und der Standardbrowser öffnet sich und zeigt die Seite an. Abbildung 11.4 stellt dies dar.

11.4.8 Automatische Erkennung von Änderungen

Das Starten eines Webserver ist eine angenehme Hilfestellung, sein volles Potential entfaltet dieser jedoch erst, wenn Änderungen an den Quelldateien erkannt und die Seite daraufhin neu geladen wird. Dies lässt sich mit einer Kombination aus Gulp *Watch-Tasks* und dem Browsersync-Reload lösen. Hierfür werden keine weiteren Module benötigt, es wird einfach ein neuer Task `watcher` in `gulpfile.mjs` angelegt und exportiert, für den Reload wird ein interner (nicht exportierter) namens `reload` erstellt. Damit die Gulp-Funktion `watch()` verfügbar ist, wird diese in der ersten Zeile ins die destrukturierende Zuweisung integriert. Ebenfalls benötigt wird `series()`, eine Funktion zum seriellen Ausführen von Tasks. Auch sie wird hinzugefügt.

```
1 import {src, dest, watch, series } from "gulp";
2
3 function reload(done) {
4   bs.reload();
5   done();
6 }
7
8 export function watcher() {
9   watch("_scss/**/*.scss", series(styles, reload));
10  watch("_js/**/*.js", series(scripts, reload));
11  watch("/*.html", reload);
```

```
12 }
```

Über den Aufruf von `watch()` können Änderungen im Dateisystem beobachtet und bei einer solchen Änderung Dinge aufgerufen werden. Der erste Parameter ist das zu beobachtende Verzeichnis, der zweite definiert die Funktion, der bei einer Änderung aufgerufen werden soll.

Der erste Aufruf in Zeile 9 beobachtet alle `.scss`-Dateien im Ordner `_scss` und allen Unterverzeichnissen. Da Änderungen an einzelnen SCSS-Dateien vorgenommen werden, genügt es hier nicht mehr, nur `main.scss` zu beobachten, sondern alle Dateien. Wenn an einer dieser Dateien eine Änderung festgestellt wurde, wird eine Serie von zwei Tasks ausgeführt: zunächst wird „styles“ aufgerufen, das den Sass-Präprozessor und den Minifier ausführt und die aktualisierte CSS-Datei generiert, im Anschluss wird mittels `reload` der Browserinhalt neu geladen. In diesem internen Hilfstask wird `bs.reload()` aufgerufen, `done()` gibt wieder an, dass die Ausführung behindert ist und Gulp mit anderen Tätigkeiten fortfahren kann.

Beim zweiten Aufruf in Zeile 10 geschieht dasselbe mit den Scripts im Verzeichnis `_js`, die bei einer Änderung erneut minimifiziert werden müssen. Dies übernimmt der Task `scripts`, im Anschluss wird der Browser neu geladen.

Der dritte Aufruf in Zeile 11 beobachtet alle HTML-Dateien im Wurzelverzeichnis. Ändert sich etwas an ihnen wird ebenfalls der `reload`-Task ausgeführt.

Somit kann nun an den CSS-, JavaScript- und HTML-Dateien gearbeitet werden und Änderungen werden automatisch im Browser reflektiert.

11.4.9 Abläufe im Default Task kombinieren

Schließlich können all diese Abläufe noch in einem Task zusammengefasst werden. Wird dieser `default` genannt, so kann er ohne Angabe eines Tasknamens einfach nur durch Eingabe von `gulp` gestartet werden. In diesem Fall werden zunächst die Tasks `styles` und `scripts` ausgeführt, um die CSS- und JavaScript-Dateien zu erzeugen, danach wird der Testserver gestartet und schließlich der Watch-Task, um Änderungen nachzuverfolgen.

Neben der zuvor bereits demonstrierten Funktion `series()` bietet Gulp auch noch `parallel()` an. Diese ermöglicht es, Tasks parallel ablaufen zu lassen. Für den Default-Task wird eine Kombination aus beidem angestrebt, da die Tasks „styles“ und „scripts“ zwar problemlos parallel laufen können, „watch“ jedoch erst ausgeführt werden sollte, wenn „server“ ausgeführt wurde und der Server erst starten sollte, wenn die CSS- und JS-Dateien erzeugt wurden.

Damit auch `parallel()` verfügbar ist, wird es beim der Destrukturierung angeführt. Eine eigene Task-Funktion muss für den Default-Task nicht erstellt werden. Beim Export werden einfach die anderen Funktionen kombiniert:

```
1 import {src, dest, watch, series, parallel} from "gulp";  
2  
3 export default series(parallel(styles, scripts), server, watcher);
```

Dieser Task wird, wie bereits erwähnt, einfach wie folgt ausgeführt:

```
> gulp
```

Somit ist ein toolgestütztes Frontend-Entwickeln möglich, das mühsame Tasks wie das Neuladen der Seite abnimmt und gleichzeitig bereits Rücksicht auf CSS-Präprozessoren und Minifizierung nimmt.

11.4.10 Dependencies ersetzen

Das Node.js-Ökosystem baut auf einer Vielzahl von Paketen auf, die zum Großteil von der Community entwickelt und gewartet werden. Im Laufe der Zeit werden in verschiedenen Node.js-Modulen Sicherheitslücken entdeckt, diese werden in der Regel auch in neuen Versionen behoben. Wird eine Bibliothek aber länger nicht aktualisiert, beruht sie weiterhin auf den alten Abhängigkeiten (Dependencies), die Sicherheitslücken aufweisen. Diese Probleme werden beim Installieren auch oft angezeigt. Weiters kann **npm audit** eingegeben werden, um einen Bericht über problematische Pakete zu erhalten.

Um nun npm zu zwingen, eine neuere Version einer Dependency zu verwenden, obwohl die beim Paket angegebenen noch eine ältere ausweisen, kann in der Datei **package.json** ein Abschnitt **overrides** spezifiziert werden. In diesem werden Paketnamen mit Versionen angegeben, die dann statt den definierten Abhängigkeiten installiert werden.

Um bei einem fiktiven Paket **some-package** Version 4.5.1 oder neuer zu erzwingen, wird folgender Abschnitt in **package.json** hinzugefügt:

```
1 "overrides": {  
2   "some-package": "4.5.1"  
3 }
```

Bei der Ersetzung von Abhängigkeiten sollte jedoch immer mit Vorsicht vorgegangen werden. Major-Versionen enthalten oft nicht rückwärtskompatible Änderungen (engl. „breaking changes“), die dann dazu führen, dass der Ablauf nicht mehr funktioniert.

Zudem werden die hier gezeigten Pakete ohnehin nur als Development-Dependencies inkludiert, d. h. sie werden nur lokal während der Entwicklung verwendet, weshalb kaum ein Risiko besteht.

11.5 Weiterführende Literatur

Bücher zum Thema Node.js sind zwar zahlreich, allerdings in der Regel schon wieder veraltet, sobald sie auf den Markt kommen, da sich das Framework mit rasenden Schritten weiterentwickelt. Wer eine Einführung sucht, der greift am Besten zu einem JavaScript-Buch wie [1], das auch eine Node.js-Einführung bietet. Ansonsten ist auch das Web voll von Tutorials und Beispielen, die beim Start helfen können.

Zum Thema CSS-Präprozessoren verhält es sich ähnlich. Bücher wie [4] bietet eine kurze Einführung in die Thematik, besser aufgehoben ist man jedoch meist mit Tutorials aus dem Web oder der offiziellen Dokumentation [3].

Zum Thema Gulp sowie Workflow-Management im Generellen gibt es jedoch eine klare Buchempfehlung. Mit [2] gibt es ein Werk, das von einem Hagenberg-Absolventen stammt und neben dem Task-Runner Gulp auch noch weitere Tools zum Workflow-Management vorstellt.

Quellenverzeichnis

- [1] Philip Ackermann. *JavaScript. Das umfassende Handbuch*. 4. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. Nov. 2025 (siehe S. 11-23).
- [2] Stefan Baumgartner. *Front-End Tooling with Gulp, Bower, and Yeoman*. Shelter Island, USA: Manning, 16. Dez. 2016 (siehe S. 11-23).
- [3] The Sass Team. *Sass: Documentation*. Jan. 2026. URL: <https://sass-lang.com/documentation> (siehe S. 11-11, 11-23).
- [4] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 11-23).
- [5] World Wide Web Consortium. *CSS Functions and Mixins Module*. W3C First Public Working Draft. 15. Mai 2025. URL: <https://www.w3.org/TR/css-mixins-1/> (siehe S. 11-10).
- [6] World Wide Web Consortium. *CSS Nesting Module Level 1*. W3C Working Draft. 22. Jan. 2026. URL: <https://www.w3.org/TR/css-nesting-1/> (siehe S. 11-9).